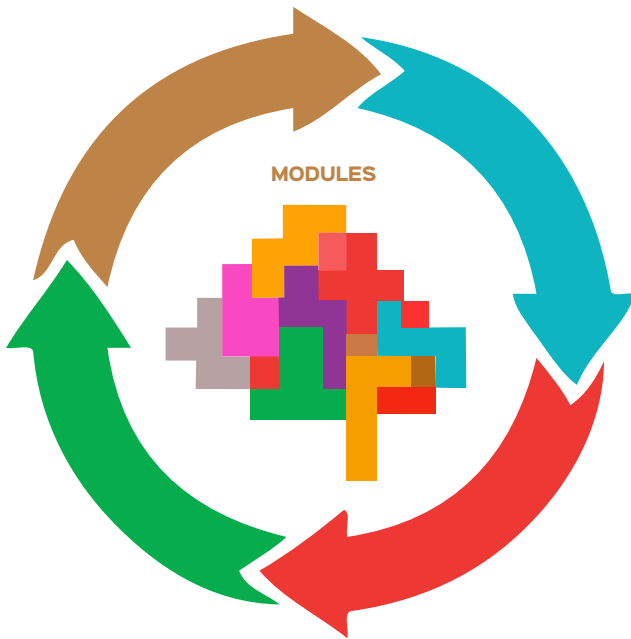


Building Reusable MODULES



The requirements on any piece of software change over a period of time. The trick is to absorb the change with no or minimal impact on the existing software. A monolith system does not offer such possibilities. Only modular systems with well-defined interfaces are flexible enough to absorb change. The Gang of Four described a good number of patterns that address the issue of modularity and named them ‘structural patterns’. We look at a few of these patterns in this second part of this series of articles on software design.

In the first part of this column (published in November 2021 issue of *Open Source For You*), we started developing a rudimentary user management system (UMS) that maintains the names and phone numbers of users. The system has only two requirements — to add user details and to retrieve them. If everything goes well, we might be able to deploy the UMS for multiple customers. In such a case, it is quite possible that some customers may want a few additional features like validation, authentication, authorisation, journaling, etc.

UMS with journaling

Let us take the case of journaling, which is a process of recording every usage of the system. It is also termed auditing. For example, this feature logs an audit record to the journal every time a new user is added to UMS.

Recollect that the original implementation of *InMemoryUserRepository* consists of only business logic. For example, the *add()* method was like this:

```
public void add(Long phone, String name) {
    entries.put(phone, name);
}
```

In order to add the journaling feature, the implementation of *add()* method can be refactored as follows:

```
public void add(Long phone, String name) {
    System.out.println("c=repository, s=add, a=" + phone +
        "&" + name);
    entries.put(phone, name);
    System.out.println("c=repository, s=add, r=void");
}
```

It can be noticed from the above snippet that the original business logic is now surrounded by a couple of logging statements. Though we are logging only to the console, it is sufficient enough for the illustration. The code that appears before the business logic is referred to as ‘pre-processing’, and that which appears after the business logic is referred to as ‘post-processing’.

The *find()* method with such pre- and post-processing code looks like this, as you might have already expected:

```
public String find(Long phone) {
    System.out.println("c=repository, s=find, a=" + phone);
    String name = entries.get(phone);
}
```